

1. Core Architecture Flow

Our system operates on a strict API-driven architecture. We do not connect to the database directly. All data requests must follow this exact 4-step sequence:

Client (Mobile/Web) → NestJS BFF (Gateway) → Supabase API (PostgREST) → PostgreSQL (RLS Protected)

- Step 1: Client (Mobile/Web): The frontend application never fetches data directly from the database. It sends standard HTTP requests to our NestJS backend.
- Step 2: NestJS BFF (Backend-For-Frontend / Gateway): This is our orchestrator. NestJS receives the frontend request, handles business logic, validates payloads, and attaches the user's JWT (Auth Token).
- Step 3: Supabase API (PostgREST): NestJS uses the Supabase SDK to send an HTTP request to the Supabase API layer. This layer translates the REST API call into a SQL query automatically.
- Step 4: PostgreSQL (RLS Protected): The underlying database receives the query from the API. Row Level Security (RLS) policies check the attached JWT. If the token is valid and has permissions, the data is returned.

2. Accessing The Database

- No Direct Database Queries & NO Traditional ORMs
 - Do not use pg, mysql2, or configure ORMs (like Prisma, TypeORM, or raw Drizzle) to connect via a database connection string.
 - Direct DB connections bypass our API gateway, break our microservices boundaries, and create security vulnerabilities.
- API Calling for ALL Tables
 - Every single database interaction (Select, Insert, Update, Delete) must be executed as an API call.
 - You must use the official `@supabase/supabase-js` SDK for all data fetching. This ensures that every request is routed through the PostgREST API layer via HTTPS.
- No API Call = No Table Access
 - If you attempt to bypass the API and query the database directly, the database will reject your request.
 - We have enabled **Row Level Security (RLS)** at the PostgreSQL level. RLS is configured to deny all traffic that does not come through the Supabase API with a valid JWT payload. There are no backdoor accesses. If your code is not making an API call, it will crash with an "Access Denied" error.

3. .env

The REST API endpoint for our database

SUPABASE_URL=[https://\[YOUR_PROJECT_REF\].supabase.co](https://[YOUR_PROJECT_REF].supabase.co)

The public key used for standard API requests (Requires RLS / JWT to read data)

SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5c...

ONLY FOR NESTJS BACKEND (Never expose to frontend): # Bypasses RLS for admin-level system tasks (e.g., cron jobs, webhooks)

SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5c...

Note:

- Do not put **DATABASE_URL** in .env
- Magbase sa api contract and sa ERD

Pakiship

pakiship-service — Supabase PostgreSQL schemas · no enforced FK constraints

parcel schema

owned by pakiship-service DB role · parcel + driver modules read/write

parcel_drafts

id (uuid)
user_id (uuid)
tracking_number
pickup_address, delivery_address
sender_name, sender_phone
receiver_name, receiver_phone
service_id, service_price
delivery_mode, assigned_driver_id
status, drop_off_point_id

parcel_draft_items

id (uuid)
parcel_draft_id (uuid)
size (S/M/L/XL)
weight_text
item_type
delivery_guarantee
quantity
photo_url
created_at

parcel_reviews ★

id (uuid)
parcel_draft_id (uuid)
reviewer_id (uuid)
rating (1–5)
review_text (500 chars)
tags []
created_at
★ not in original ERD

driver schema

owned by pakiship-service DB role · driver module reads/writes

driver_profiles

id (uuid)
vehicle_type, license_number
delivery_mode (relay/direct)
is_online, acceptance_rate
documents_status

driver_jobs

id (uuid)
job_number
parcel_draft_id (uuid)
driver_id (uuid)
status, earnings

driver_earnings ★

id (uuid)
driver_id (uuid)
job_id (uuid)
amount, earned_at
★ not in original ERD

location schema

owned by pakiship-service DB role · tracking module reads · ApingAPI adapter writes

drop_off_points

id (uuid)
name, address
operating_hours
lat, lng (6 decimal)
storage_capacity, is_active

parcel_tracking_events ★

id (uuid)
parcel_draft_id (uuid)
status, location_label
lat, lng
occurred_at

parcel_hub_records

id (uuid)
parcel_draft_id (uuid)
hub_id (uuid)
status, storage_location
received_at, dispatched_at

routing schema

owned by pakiship-service DB role · pricing + parcel modules read · ShoTeam feeds

operator_hubs

id (uuid)
owner_user_id (uuid)
code, name
address, lat, lng
storage_capacity
is_active, geofence_on

operator_earnings

id (uuid)
hub_id (uuid)
parcel_draft_id (uuid)
parcel_hub_record_id (uuid)
earning_type, amount
currency, earned_at

operator_incentives

id (uuid)
hub_id (uuid)
incentive_type
title, amount
awarded_at, notes

all references are plain uuid columns — no enforced FK constraints. Integrity enforced at API layer. ★ = not in original ERD

Pakipark

pakipark-service — Supabase PostgreSQL schemas · no enforced FK constraints

reservation schema

owned by pakipark-service DB role · reservation module reads/writes

bookings

id (uuid)
user_id (uuid)
vehicle_id (uuid)
location_id (uuid)
parking_slot_id (uuid)
reference, barcode (unique)
spot, date, timeSlot, type
status, amount, paymentMethod
paymentStatus, checkInAt, checkOutAt

transaction_logs

id (uuid)
booking_id (uuid)
user_id (uuid)
type, amount
details (jsonb)
createdAt, updatedAt
★ No refund/escrow fields yet.
GREENOVATE escrow boundary.

parking_lot schema

owned by pakipark-service DB role · parking-lot module reads/writes

locations

id (uuid)
name, address
lat, lng, coordinates (jsonb)
totalSpots, availableSpots
hourlyRate, pricePerHour
imageUrl, operatingHours (jsonb)
amenities [], owner_id (uuid)
isActive, createdAt
★ hourlyRate + pricePerHour
duplicate — resolve before migration
★ owner_id unique = 1 partner
per location only

parking_slots

id (uuid)
location_id (uuid)
label, section, floor
type (regular/disabled/ev)
status (available/occupied)

parking_rates

id (uuid)
location_id (uuid)
type (hourly/daily/monthly)
rate, createdAt, updatedAt
ShoTeam pricing boundary

partner schema

owned by pakipark-service DB role · partner module reads/writes

partner identity

No local table.
owner_id resolved via
internal API call to
shared-service auth

reviews

id (uuid)
user_id (uuid)
location_id (uuid)
booking_id (uuid, unique)
rating (1-5), comment

activity_logs

id (uuid)
admin_id (uuid)
action, targetType
target_id (uuid)
details (jsonb)

teller schema

owned by pakipark-service DB role · teller module reads/writes

vehicles

id (uuid), user_id (uuid)
brand, model, plateNumber

uploads

id (uuid), user_id (uuid)
entity_id (uuid), url

settings

id (uuid), key (unique)
value (jsonb)

n uuid columns — no enforced FK constraints. Integrity enforced at API layer. ★ = design issues to flag before migration

Shared-services - “Account” schema yung auth schema

shared-service :3000 — Supabase PostgreSQL schemas · no enforced FK constraints
Owner: Joaquin / Aeus · consumed by pakiship-service, pakipark-service, pakiAdmin

auth schema

identity, roles, sessions · read by all services · written only by shared-service

profiles

id (uuid)
full_name, email
phone, dob
role (customer/driver/
operator/partner/teller/
admin)
address, city, province
profile_picture
is_verified, documents
two_factor_enabled
notification_preferences
created_at

admin_accounts

id (uuid)
profile_id (uuid)
admin_role (superadmin/
ops_staff/data_analyst/
compliance_officer)
permissions (jsonb)
is_active
last_login_at
created_at
★ Backoffice + Data Team
from the admin dashboard

document_verifications

id (uuid)
profile_id (uuid)
document_type
(driver_license/
business_permit/
land_use/or_cr)
file_url
status (pending/verified/
rejected)
rejection_reason
reviewed_by (uuid)
reviewed_at

notifications schema

cross-service notifications · written by all services via shared-service API · read by clients

notifications

id (uuid)
user_id (uuid)
type (delivery/parking/system/promo)
title, message
is_read
source_service (pakiship/pakipark)
created_at

announcements

id (uuid)
title, message
target_role (all/customer/
driver/operator/partner)
target_service (pakiship/
pakipark/all)
created_by (uuid), expires_at

payment schema

financial ledger · written only by shared-service payment/ module · GREENOVATE + ApingAPI adapter boundary

payment_transactions

id (uuid)
user_id (uuid)
source_service
(pakiship/pakipark)
reference_id (uuid)
(parcel_draft_id or
booking_id)
amount, currency
payment_method
gateway_transaction_id
hold_id (escrow)
status (pending/captured/
refunded/failed)
created_at, captured_at
refunded_at

escrow_holds

id (uuid)
transaction_id (uuid)
user_id (uuid)
hold_amount
hold_id (ApingAPI ref)
status (held/released/
partially_released/
forfeited)
held_at
release_at (deadline)
released_at
release_reason
GREENOVATE escrow
adapter boundary

refunds

id (uuid)
transaction_id (uuid)
user_id (uuid)
refund_amount
reason
gateway_refund_id
status (pending/
processed/failed)
initiated_by (uuid)
(admin or system)
created_at
processed_at
50% no-show rule
enforced here

All references are plain uuid columns — no enforced FK constraints · Joaquin signs off on all migrations to this service

